

Sakum Lang - Pipe Operator (|>)

pravah | Functional data-flow for Hinglish systems code | Generated from live system

1. What is the Pipe Operator?

The pipe operator (`|>`) - called `pravah` in Hinglish - passes the result of the expression on its left as the **FIRST** argument to the function call on its right. It is borrowed from Elixir's `|>` and enables readable, left-to-right data transformation pipelines instead of deeply nested function calls.

```
# Without pipe (nested, hard to read):
lek(guna(jodo(5, 3), 2))    # prints 16

# With pipe (flat, left-to-right):
5 |> jodo 3 |> guna 2 |> lek    # prints 16
```

2. Semantics

```
a |> f()          -> f(a)
a |> f(b, c)      -> f(a, b, c)
a |> f() |> g()   -> g(f(a))
a |> f() |> g() |> h() -> h(g(f(a)))
```

The piped value always becomes the **FIRST** argument. Additional arguments on the right function follow in order. Chains are left-associative and evaluated innermost-left first.

3. Precedence (Elixir-compatible)

The pipe operator binds **LOOSER** than arithmetic. Level 170 vs `+` `-` at 210 and `*` `/` at 220. This means arithmetic groups **FIRST**, then the pipe applies:

```
5 + 3 |> double()    -> (5 + 3) |> double() -> double(8) = 16
2 * 3 |> double()    -> (2 * 3) |> double() -> double(6) = 12
5 |> jodo 3 |> guna 2 -> guna(jodo(5,3), 2) -> 16
```

4. Usage Patterns

```
# Basic pipe
naam r1 = 5 |> double()
lek(r1)                                # 10

# Pipe with extra args
naam r2 = 10 |> add(5)
lek(r2)                                # 15

# Chained pipe
naam r3 = 3 |> double() |> add(1)
lek(r3)                                # 7

# Multi-line pipe chain (indent 4 spaces)
naam r4 = 2
    |> double()
    |> triple_add(1, 2)
lek(r4)                                # 7

# Pipe in arithmetic expression
naam r5 = 5 + 3 |> double()
lek(r5)                                # 16
```

5. Survival Rules (do not break these)

- RULE 1: Indent pipes 4 spaces deeper than the chain start (see r4 example above).
- RULE 2: Pipe result must be used - assign (naam x = ...) or output (lek(...)).
- RULE 3: Right side must be a function call or identifier resolving to a function.
- RULE 4: Avoid >3 pipes in a single chain; extract a kriya (function) instead.
- RULE 5: Never start a statement with |> (left operand required).
- RULE 6: Pipe binds looser than + - * / ; wrap left in () if you need it grouped.

6. AI-Powered Survival Suggestions

The Mantra REPL auto-suggests the next survival code after every pipe / naam / kriya line. It checks indentation, parenthesis balance, pipe-chain depth, and unused results - then queries the SAKUM AI neuro-core and the Knowledge base for pipe patterns.

```
# In REPL, type:
mantra> suggest 5 |> double() |>

# Output:
=== Survival Code Suggestions ===
  Pipe chain active
  Next indent: 4 spaces
  Continue pipe chain: add function call after |>
  AI note: Pipe (|>) / pravah: passes left value as first arg
            to right function. Use: value |> func() or
            value |> func(arg1, arg2).
  Pipe-friendly funcs: jodo, ghata, guna, bhaga, lek, double...
```

7. Cross-Platform Implementation

SAKUM_HINGLISH.md	pravah keyword registry
SAKUM_LANG.md	Pipe operator spec + precedence
mantra.py	Lexer + parser + eval + suggestion
sutra_ir.py	Opcode.PIPE + IRBuilder.pipe()
sakum_pipeline.s	x86-64 lexer/parser/codegen
sakum_eval.s	x86-64 bootstrap evaluator
sakum_ai.s	Neuro-core (queries Knowledge/)
examples/pipe.sak	Worked examples + self-test

8. Quick Reference

```
Operator:    |>   (Hinglish: pravah)
Meaning:     left_result becomes 1st arg of right fn
Associativity: left
Precedence:  level 170 (looser than + - * /)
Chaining:    a |> f() |> g() |> h()
Multi-line:  indent continuation 4 spaces
Suggestion:  mantra> suggest <code>
```